FitPocket

Documentación técnica del proyecto

Stack: Vite + React + JS | n8n | JSON local

Versión 1.0 — 2026

1. Visión general del proyecto

FitPocket es una aplicación web progresiva (PWA) orientada a personas que comienzan en el gimnasio sin conocimiento previo. Permite generar rutinas de entrenamiento personalizadas, equilibrar la alimentación según la complexión del usuario y hacer seguimiento visual del progreso a lo largo del tiempo.

1.1 Objetivos principales

- Generar rutinas de gimnasio adaptadas al perfil físico del usuario (peso, altura, edad, objetivo).
- Balancear una dieta calculando TDEE y macronutrientes según la fórmula Mifflin-St Jeor.
- Visualizar el progreso corporal con gráficos de peso, racha de entrenamientos y adherencia semanal.
- Funcionar como PWA instalable en dispositivos móviles (iOS y Android) sin necesidad de una app nativa.
- Almacenar todos los datos en archivos JSON locales, sin depender de un servidor de base de datos externo.

1.2 Principios de diseño técnico

- Simplicidad ante todo: las decisiones de stack priorizan velocidad de desarrollo y curva de aprendizaje baja.
- Sin backend propio: n8n reemplaza completamente a un servidor Express o similar.

- Datos en JSON local: lectura y escritura directa desde n8n, sin motor de base de datos.
- Offline-first: la PWA cachea los datos necesarios para funcionar sin conexión.
- Separación de responsabilidades: React solo renderiza UI, n8n procesa toda la lógica de negocio.

---

2. Arquitectura del sistema

El sistema se divide en tres capas bien definidas que se comunican entre sí de forma clara y predecible:

Capa	Responsabilidad	Tecnologías
Frontend	Interfaz de usuario, formularios, visualización de datos	Vite, React 18, JS
Automatización	Lógica de negocio, integraciones, cálculos, notificaciones	n8n
Persistencia	Almacenamiento de datos estructurados en disco	Archivos JSON locales

El frontend nunca accede directamente a los archivos JSON. Toda operación de lectura o escritura pasa por un webhook de n8n, que actúa como única puerta de entrada a los datos.

2.1 Flujo de datos

- El usuario interactúa con la UI de React (completa un formulario, marca un entrenamiento, solicita una rutina).
- React ejecuta un POST al webhook correspondiente de n8n a través del cliente centralizado n8nClient.js.
- n8n procesa la solicitud: valida, calcula, llama a APIs externas si es necesario y lee o escribe el archivo JSON.
- n8n responde con un JSON que React recibe, cachea con React Query y muestra en pantalla.
- Los cron jobs de n8n corren de forma autónoma cada mañana para actualizar métricas y enviar notificaciones.

---

3. Capa frontend

3.1 Vite 5

Vite es la herramienta de build y servidor de desarrollo del proyecto. Reemplaza a Create React App con una experiencia significativamente más rápida gracias al uso de ESModules nativos del navegador durante el desarrollo.

Por qué Vite

- Arranque del servidor de desarrollo en menos de 300ms independientemente del tamaño del proyecto.
- Hot Module Replacement (HMR) instantáneo: los cambios en el código se reflejan en el navegador sin recargar la página.
- Build de producción basado en Rollup, altamente optimizado con tree-shaking y code splitting automático.
- Soporte nativo para variables de entorno con el prefijo VITE_, accesibles via import.meta.env.
- Selección automática de .env.development o .env.production según el comando ejecutado.

Comandos principales

```
npm create vite@latest fitpocket -- --template react
npm run dev          # servidor de desarrollo en localhost:5173
npm run build        # build de producción en /dist
npm run preview      # preview del build de producción
```

3.2 React 18 con JavaScript

React 18 es la biblioteca principal de UI. El proyecto usa JavaScript puro (sin TypeScript) para reducir la fricción inicial y acelerar el desarrollo del MVP.

Funcionalidades de React 18 utilizadas

- Concurrent Features: Suspense y useTransition para cargar datos sin bloquear la UI mientras n8n responde.
- Hooks estándar: useState para estado local de formularios, useEffect para suscripciones y cleanup, useCallback y useMemo para optimizaciones puntuales.
- Context API para el estado de autenticación del usuario (datos del perfil disponibles globalmente).
- Lazy loading con React.lazy() y Suspense para cargar cada módulo solo cuando el usuario lo visita.

Ejemplo de lazy loading por módulo

```
const Rutinas  = React.lazy(() => import('./pages/Rutinas'));
```

```
const Dieta    = React.lazy(() => import('./pages/Dieta'));
const Progreso = React.lazy(() => import('./pages/Progreso'));
```

3.3 React Router v6

Maneja la navegación entre las cuatro secciones principales de la aplicación. Se usa el modo hash para compatibilidad con deployments estáticos.

Ruta	Componente	Descripción
/perfil	Perfil.jsx	Datos personales, complexión, objetivos
/rutinas	Rutinas.jsx	Generador y visualizador de rutinas
/dieta	Dieta.jsx	Calculadora de macros y plan de comidas
/progreso	Progreso.jsx	Gráficos de peso, racha y adherencia
/logros	Logros.jsx	Badges desbloqueados por el usuario

Rutas protegidas

Todas las rutas excepto /login están envueltas en un componente RutaProtegida que verifica si el usuario tiene perfil creado. Si no, redirige a /perfil para completar los datos antes de usar la app.

3.4 Zustand — estado global

Zustand es la biblioteca de estado global del proyecto. Se eligió sobre Redux por su API minimalista, sin boilerplate y perfectamente compatible con JavaScript sin tipado.

Stores definidos

- usePerfilStore: datos del usuario (peso, altura, edad, sexo, objetivo, nivel de actividad).
- useRutinaStore: rutina activa generada por n8n, con la lista de días y ejercicios.
- useDietaStore: plan de comidas activo con distribución calórica y macros por comida.
- useProgresoStore: historial de registros de peso y métricas semanales.

3.5 React Query (TanStack Query)

React Query maneja el ciclo de vida de todas las llamadas a n8n: estado de carga, caché, reintentos y revalidación. Evita llamadas repetidas a n8n cuando los datos ya están en caché.

Configuración global

```
const queryClient = new QueryClient({
  defaultOptions: {
    queries:   { staleTime: 5 * 60 * 1000, retry: 2 },
    mutations: { retry: 1 },
  }
});
```

3.6 Tailwind CSS v4 + shadcn/ui

Tailwind CSS provee el sistema de estilos utilitario. shadcn/ui agrega componentes accesibles pre-construidos (modales, formularios, sliders, toasts) que se copian al proyecto y se pueden modificar libremente.

Componentes de shadcn/ui usados en FitPocket

- Slider: para seleccionar peso y altura en el formulario de perfil.
- Select: para elegir objetivo, nivel de actividad y grupo muscular.
- Dialog: para mostrar el detalle de un ejercicio con su GIF animado.
- Progress: barra de progreso hacia el peso objetivo.
- Toast: notificaciones de acciones completadas (entreno registrado, badge desbloqueado).

3.7 Recharts — visualización de progreso

Recharts es la biblioteca de gráficos basada en D3 y React. Se usa para los tres gráficos principales del módulo de progreso:

Gráfico	Tipo	Datos mostrados
Evolución de peso	LineChart	Peso registrado por fecha vs meta
Adherencia semanal	BarChart	Porcentaje de días entrenados por semana
Distribución macros	PieChart	Proteínas / carbohidratos / grasas del día

3.8 Framer Motion — animaciones

Se usa para micro-animaciones que mejoran la experiencia en móvil: aparición de tarjetas de ejercicios, animación de badges al desbloquearse y transiciones entre rutas. Las animaciones

están condicionadas a prefers-reduced-motion para respetar las preferencias del sistema operativo del usuario.

3.9 PWA con Vite PWA Plugin

La aplicación es instalable como PWA en iOS y Android sin necesidad de publicarla en ninguna tienda. El plugin vite-plugin-pwa genera automáticamente el Service Worker y el manifest de la aplicación.

Capacidades habilitadas por la PWA

- Instalación en la pantalla de inicio de cualquier dispositivo móvil o desktop.
- Funcionamiento offline: React Query cachea los últimos datos recibidos y la app los muestra sin conexión.
- Web Push Notifications: recordatorios de entrenamiento y notificaciones de logros sin abrir el navegador.
- Splash screen y modo fullscreen: la app se ve y se comporta como nativa al abrirla desde el ícono.

---

4. Capa de automatización — n8n

n8n es la pieza central del backend de FitPocket. Reemplaza completamente a un servidor Express o similar: toda la lógica de negocio, los cálculos, las integraciones con APIs externas y las operaciones sobre los archivos JSON se implementan como flujos visuales en n8n.

El frontend nunca sabe de dónde vienen los datos ni cómo se calcularon. Solo conoce la URL del webhook y el formato del JSON que recibe como respuesta.

4.1 Rol de n8n en el proyecto

Responsabilidad	Cómo se implementa en n8n
Recibir solicitudes del frontend	Nodo Webhook (POST con Header Auth)
Validar y transformar datos	Nodo Function (JavaScript puro)
Calcular TDEE, IMC, macros	Nodo Function con fórmulas Mifflin-St Jeor
Consultar APIs externas	Nodo HTTP Request (ExerciseDB, Nutritionix, OpenAI)
Leer y escribir archivos JSON	Nodo Read/Write Binary File + nodo Function

Responsabilidad	Cómo se implementa en n8n
Lógica condicional	Nodo IF con condiciones múltiples (AND / OR)
Iterar sobre colecciones	Nodo Split in Batches + Loop Over Items
Tareas programadas	Nodo Schedule Trigger (cron expressions)
Enviar notificaciones	Nodo HTTP Request a Web Push / SendGrid
Responder al frontend	Nodo Respond to Webhook

4.2 Flujos implementados

Flujo 1 — generación de rutina

Disparado por: webhook POST /rutina desde React.

- Recibe peso, altura, edad, sexo, objetivo, músculo objetivo y días por semana.
- Calcula IMC y nivel de dificultad (principiante / intermedio / avanzado) con un nodo Function.
- Nodo IF separa el flujo: principiantes van a ExerciseDB con filtro de dificultad baja, el resto puede ir a OpenAI para personalización mayor.
- Nodo Merge une las ramas y un segundo Function arma el JSON de la rutina por días.
- Guarda el resultado en usuarios/{user_id}/rutina.json.
- Responde con el JSON de la rutina al frontend.

Flujo 2 — balance de dieta

Disparado por: webhook POST /dieta desde React.

- Recibe peso, altura, edad, sexo, objetivo y nivel de actividad.
- Aplica la fórmula Mifflin-St Jeor para calcular el TMB y multiplica por el factor de actividad para obtener el TDEE.
- Nodo IF ajusta las calorías: déficit de 300 kcal para perder peso, superávit de 300 kcal para ganar músculo, mantenimiento sin ajuste.
- Consulta Nutritionix API para obtener sugerencias de alimentos concretos.
- Distribuye calorías y macros en cuatro comidas (desayuno 25%, almuerzo 35%, merienda 15%, cena 25%).
- Guarda en usuarios/{user_id}/dieta.json y responde al frontend.

Flujo 3 — actualización de progreso (cron)

Disparado por: Schedule Trigger todos los días a las 8:00 AM.

- Cron expression: 0 8 * * *
- Lee el archivo usuarios/index.json para obtener la lista de usuarios activos.
- Nodo Split in Batches procesa cada usuario individualmente.
- Lee el archivo de registros de cada usuario y calcula racha de días consecutivos, adherencia semanal y tendencia de peso.
- Nodo IF detecta si se alcanzó un logro (racha >= 7 días, meta de peso alcanzada, etc.).
- Rama true: registra el badge y envía notificación Web Push o email.
- Rama false: envía recordatorio estándar si la adherencia es menor al 70%.
- Actualiza usuarios/{user_id}/progreso.json con las métricas recalculadas.

Flujo 4 — logros y badges

Disparado por: webhook POST /entreno-completo desde React, o llamado como sub-flujo desde el cron.

- Recibe user_id, fecha y peso actual.
- Lee usuarios/{user_id}/badges.json para no repetir logros ya otorgados.
- Evalúa el catálogo completo de logros contra el estado actual del usuario.
- Agrega los logros nuevos al archivo badges.json.
- Envía notificación inmediata si el flujo fue disparado por el webhook (experiencia en tiempo real).

4.3 Comunicación React ↔ n8n

Toda comunicación se centraliza en el archivo src/lib/n8nClient.js. Este módulo expone una función n8n(endpoint, body) que agrega automáticamente los headers de autenticación y maneja errores y timeouts.

Autenticación

Los webhooks de n8n están protegidos con Header Authentication. React envía el header x-api-key con un valor secreto almacenado en la variable de entorno VITE_N8N_API_KEY. Esta clave nunca se incluye en el repositorio.

Manejo de errores

- Error 401: credenciales inválidas. Se muestra mensaje de sesión expirada.
- Error 422: datos incompletos o inválidos. Se muestra qué campo falta.
- Error 5xx: fallo en n8n o en una API externa. Se muestra mensaje de reintento.
- AbortError: timeout de 15 segundos. Se informa al usuario que la solicitud tardó demasiado.

4.4 Hosting de n8n

Opción	Costo	Uso recomendado
Docker local	Gratuito	Desarrollo y testing en la computadora del equipo
Railway / Render	Gratuito (tier básico)	MVP y demos del proyecto
n8n Cloud	Pago mensual	Producción con soporte oficial

Comando para correr n8n con Docker

```
docker run -it --rm --name n8n -p 5678:5678 \
  -v ~/.n8n:/home/node/.n8n \
  docker.n8n.io/n8nio/n8n
```

El webhook queda disponible en http://localhost:5678/webhook/{endpoint} durante el desarrollo local.

---

5. Capa de persistencia — archivos JSON locales

FitPocket almacena todos sus datos en archivos JSON planos en el sistema de archivos donde corre n8n. Esta decisión elimina la necesidad de instalar y mantener un motor de base de datos, lo que simplifica radicalmente el setup inicial del proyecto.

5.1 Estructura de carpetas

```
data/
├── usuarios/
│   ├── index.json              # lista de todos los usuarios
│   └── {user_id}/
│       ├── perfil.json         # datos de complexión y preferencias
│       ├── rutina.json         # última rutina generada
│       ├── dieta.json          # último plan de dieta
│       ├── progreso.json       # métricas semanales actuales
│       ├── registros.json      # historial de entrenamientos
│       └── badges.json         # logros desbloqueados
└── config/
    ├── ejercicios_cache.json   # caché de ExerciseDB para evitar llamadas repetidas
    └── alimentos_cache.json    # caché de Nutritionix
```

5.2 Estructura de cada archivo

perfil.json

```
{
  "user_id":        "a1b2c3d4",
  "nombre":         "Juan",
  "email":          "juan@email.com",
  "peso":           75,
  "altura":         175,
  "edad":           25,
  "sexo":           "masculino",
  "objetivo":       "hipertrofia",
  "nivel_actividad":"moderado",
  "meta_peso":      80,
  "push_token":     "...",
  "creado_en":      "2026-01-01T00:00:00Z"
}
```

registros.json

```
{
  "registros": [
    { "fecha": "2026-03-25", "completado": true, "peso": 74.5 },
    { "fecha": "2026-03-24", "completado": true, "peso": 74.8 }
  ]
}
```

badges.json

```
{
  "badges": [
    { "tipo": "primer_entreno", "titulo": "Primer paso", "ganado_en": "2026-01-05T..."
},
    { "tipo": "racha_7",        "titulo": "Semana perfecta", "ganado_en":
"2026-01-12T..." }
  ]
}
```

5.3 Operaciones desde n8n

n8n lee y escribe archivos JSON usando los nodos Read/Write Binary File combinados con nodos Function para parsear y serializar. El patrón es siempre el mismo:

Leer un archivo

```
// Nodo Read Binary File
// Path: /data/usuarios/{{ $json.user_id }}/registros.json

// Nodo Function posterior
const contenido = Buffer.from($binary.data.data, "base64").toString("utf8");
const datos = JSON.parse(contenido);
return [{ json: datos }];
```

Escribir un archivo

```
// Nodo Function — preparar datos
const registros = $input.first().json.registros;
registros.push({ fecha: $json.fecha, completado: true, peso: $json.peso });
const contenido = JSON.stringify({ registros }, null, 2);
const buffer = Buffer.from(contenido, "utf8").toString("base64");
return [{ json: {}, binary: { data: { data: buffer, mimeType: "application/json" } }
}];

// Nodo Write Binary File posterior
// Path: /data/usuarios/{{ $json.user_id }}/registros.json
```

5.4 Consideraciones importantes

Ventajas para este proyecto

- Setup inmediato: no requiere instalar ni configurar ningún servicio externo.
- Datos legibles: cualquier integrante del equipo puede abrir un archivo .json y ver el estado de la aplicación.
- Sin migraciones: cambiar la estructura de datos es tan simple como editar el archivo.
- Portabilidad total: los datos se pueden copiar, versionar con Git o mover entre entornos con un simple zip.

Limitaciones a tener en cuenta

- Sin consultas avanzadas: no hay equivalente a SQL WHERE o índices. Los filtros se hacen en JavaScript dentro de los nodos Function.
- Concurrencia limitada: si dos usuarios generan una rutina al mismo tiempo y n8n escribe el mismo archivo, puede haber conflictos. Para el MVP con pocos usuarios simultáneos esto no es un problema.

- Sin relaciones: los datos entre archivos se relacionan manualmente por user_id. No hay foreign keys ni joins.
- Escalabilidad: esta solución es adecuada para un proyecto académico o MVP. Si el proyecto crece a producción real con muchos usuarios, se recomienda migrar a MongoDB Atlas o PocketBase manteniendo la misma estructura de documentos JSON.

---

6. APIs externas integradas

Todas las integraciones con APIs externas se realizan desde n8n usando el nodo HTTP Request. El frontend nunca llama directamente a estas APIs, lo que mantiene las claves seguras en el servidor de n8n.

API	Uso en FitPocket	Autenticación	Costo
ExerciseDB (RapidAPI)	Obtener ejercicios por grupo muscular y nivel	RapidAPI Key (header)	Gratuito hasta 10 req/día
Nutritionix	Sugerencias de alimentos y valores nutricionales	App ID + App Key (header)	Gratuito hasta 500 req/día
OpenAI GPT-4o-mini	Rutinas avanzadas personalizadas con IA	Bearer token	Pago por token
Web Push (VAPID)	Notificaciones push a dispositivos	VAPID keys	Gratuito
SendGrid	Emails de logros y resúmenes semanales	API Key (header)	Gratuito hasta 100 emails/día

Las claves de todas estas APIs se almacenan en las Credenciales de n8n (sección Credentials del panel), nunca en el código ni en variables de entorno del frontend.

---

7. Estructura de carpetas del proyecto

```
fitpocket/
├── public/
│   ├── icons/                  # íconos PWA en múltiples tamaños
│   └── manifest.webmanifest    # config de la PWA
├── src/
│   ├── components/             # componentes reutilizables
│   │   ├── ui/                 # componentes de shadcn/ui
│   │   ├── EjercicioCard.jsx
│   │   ├── MacroRing.jsx
```

```
│   │   ├── BadgeItem.jsx
│   │   └── EstadoPeticion.jsx
│   ├── pages/                  # una carpeta por módulo
│   │   ├── Perfil.jsx
│   │   ├── Rutinas.jsx
│   │   ├── Dieta.jsx
│   │   ├── Progreso.jsx
│   │   └── Logros.jsx
│   ├── hooks/                  # custom hooks con React Query
│   │   ├── useRutina.js
│   │   ├── useDieta.js
│   │   ├── useProgreso.js
│   │   ├── useLogros.js
│   │   └── useCompletarEntreno.js
│   ├── stores/                 # stores de Zustand
│   │   ├── usePerfilStore.js
│   │   ├── useRutinaStore.js
│   │   └── useDietaStore.js
│   ├── lib/
│   │   └── n8nClient.js        # cliente centralizado para webhooks
│   ├── App.jsx                 # rutas y layout principal
│   └── main.jsx                # punto de entrada
├── data/                       # archivos JSON (en el servidor de n8n)
├── .env.development
├── .env.production
├── .env.example
├── vite.config.js
└── package.json
```

---

8. Dependencias del proyecto

8.1 Dependencias de producción

Paquete	Versión	Uso
react	18.x	Biblioteca principal de UI
react-dom	18.x	Renderizado en el navegador
react-router-dom	6.x	Navegación entre páginas
@tanstack/react-query	5.x	Caché y estado de llamadas a n8n

Paquete	Versión	Uso
zustand	4.x	Estado global liviano
recharts	2.x	Gráficos de progreso
framer-motion	11.x	Animaciones y transiciones
tailwindcss	4.x	Sistema de estilos utilitario

8.2 Dependencias de desarrollo

Paquete	Versión	Uso
vite	5.x	Build tool y servidor de desarrollo
@vitejs/plugin-react	4.x	Soporte de JSX y Fast Refresh
vite-plugin-pwa	0.x	Generación de Service Worker y manifest
eslint	9.x	Linting del código JavaScript
prettier	3.x	Formateo automático de código
vitest	1.x	Tests unitarios compatible con Vite

Instalación completa

```
npm install react react-dom react-router-dom @tanstack/react-query zustand recharts
framer-motion
npm install -D vite @vitejs/plugin-react vite-plugin-pwa tailwindcss eslint prettier
vitest
```

---

9. Variables de entorno

Variable	Entorno	Descripción
VITE_N8N_BASE_URL	dev + prod	URL base del servidor n8n
VITE_N8N_API_KEY	dev + prod	Clave secreta para autenticar los webhooks

Valores por entorno

```
# .env.development
```

```
VITE_N8N_BASE_URL=http://localhost:5678
VITE_N8N_API_KEY=clave-para-desarrollo
```

```
# .env.production
VITE_N8N_BASE_URL=https://tu-n8n.railway.app
VITE_N8N_API_KEY=clave-secreta-de-produccion
```

Agregar .env y .env.development y .env.production al archivo .gitignore. Solo subir .env.example con las claves vacías.

---

10. Deploy y entornos

Capa	Servicio	Proceso
Frontend (React + Vite)	Vercel o Netlify	Conectar repositorio GitHub, push a main dispara el build automáticamente
n8n (automatización)	Railway o Render	Deploy desde imagen Docker oficial de n8n, configurar variables de entorno en el panel
Datos JSON	Volumen persistente de Railway/Render	Montar el directorio /data como volumen para que los JSON sobrevivan reinicios

10.1 Checklist de deploy

- Configurar VITE_N8N_BASE_URL y VITE_N8N_API_KEY en las variables de entorno de Vercel/Netlify.
- Configurar las credenciales de APIs externas (ExerciseDB, Nutritionix, SendGrid) en el panel de Credenciales de n8n.
- Activar todos los flujos de n8n (por defecto quedan inactivos al importarlos).
- Verificar que el volumen de datos de Railway/Render esté montado correctamente antes del primer uso.
- Probar cada webhook manualmente desde el panel de n8n antes del lanzamiento.
- Habilitar HTTPS en todos los endpoints (Vercel y Railway lo hacen automáticamente).